

รายงานโครงงาน

PrimeSkill: เว็บแอปพลิเคชันทะเบียนและตลาดกลาง สำหรับเครื่องมือและปลั๊กอินของ AI Agent

จัดทำโดย

ณภัทร อรัญพูล 673380036-3

ธนินธร อันทรบุตร 673380043-6

ศุภกร กรมรินทร์ 673380061-4

ณัชพล เพ็งพล 673380267-4

ณัฐกรณ์ อินธิสาร 673380268-2

เสนอ

รศ. ดร.ปัญญาพล หอระตะ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา CP353002

Principles of Software Design and Development สาขาวิทยาการคอมพิวเตอร์

คณะวิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

บทคัดย่อ

รายงานฉบับนี้นำเสนอการวิเคราะห์ ออกแบบ และพัฒนา PrimeSkill ซึ่งเป็นเว็บแอปพลิเคชันทะเบียนและตลาดกลางสำหรับเครื่องมือและปลั๊กอินของ AI agent ที่พัฒนาขึ้นเพื่อแก้ปัญหาที่เครื่องมือเหล่านี้กระจายอยู่ตามที่เก็บซอร์สโค้ดและเว็บไซต์ต่าง ๆ ทำให้ค้นหาได้ยาก ขาดตัวชี้วัดคุณภาพ ขาดการจัดการเวอร์ชัน และไม่มีขั้นตอนตรวจสอบก่อนเผยแพร่ ระบบให้นักพัฒนาลงทะเบียนและเผยแพร่ข้อมูลและเวอร์ชันของเครื่องมือ ให้ผู้ดูแลระบบ (ADMIN) ตรวจสอบและอนุมัติก่อนเผยแพร่ตามวงจร DRAFT ⟶ PENDING ⟶ PUBLISHED ⟶ DEPRECATED และให้ผู้ใช้ค้นหา กรองตามหมวดหมู่และแท็ก จัดเรียง ให้คะแนนและเขียนรีวิวได้

ระบบพัฒนาด้วย Java 17 และ Spring Boot มีทั้งหน้าเว็บ Thymeleaf และ REST API ที่มีเอกสาร OpenAPI/Swagger ใช้ Spring Data JPA เชื่อมต่อฐานข้อมูล PostgreSQL (Supabase) และใช้ Spring Security ควบคุมการยืนยันตัวตนด้วย session การเข้ารหัสรหัสผ่านด้วย BCrypt การป้องกัน CSRF และสิทธิ์ตามบทบาท USER/ADMIN ออกแบบด้วยสถาปัตยกรรมแบบแบ่งชั้น (Controller–Service–Repository) แยก DTO/Mapper ออกจาก entity ใช้ constructor injection และประยุกต์ GoF Behavioral Patterns สามแบบ ได้แก่ Strategy สำหรับการเรียงลำดับผลค้นหา State สำหรับวงจรการเผยแพร่ และ Observer สำหรับบันทึก audit หลังบันทึกรีวิว ฐานข้อมูลมี 8 ตารางธุรกิจ ครอบคลุมความสัมพันธ์แบบ one-to-one, one-to-many และ many-to-many

ผลการดำเนินงานบันทึกไว้ใน repository ว่าระบบผ่านการทดสอบอัตโนมัติ 736 รายการ ประกอบด้วยการทดสอบหน่วย การทดสอบ controller และสิทธิ์ และการทดสอบเชิงบูรณาการกับ PostgreSQL ณ commit ที่ระบุ รายงานยังประเมินการประยุกต์หลัก SOLID ตามหลักฐานในโค้ด พบว่ามีการประยุกต์ได้ดีในการแยกชั้นงานและการพึ่งพา interface แต่ยังมีจุดที่ควรปรับปรุง เช่น นโยบายการเรียงลำดับตามคะแนนและความเกี่ยวข้องและการแยกหน้าที่ของ service จัดการเวอร์ชัน พร้อมเสนอแนวทางพัฒนาต่อ

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อเป็นส่วนหนึ่งของรายวิชา CP353002 Principles of Software Design and Development (Spring Boot) โดยมีจุดมุ่งหมายเพื่ออธิบายการวิเคราะห์ ออกแบบ และพัฒนา PrimeSkill ซึ่งเป็นเว็บแอปพลิเคชันทะเบียนและตลาดกลางสำหรับเครื่องมือและปลั๊กอินของ AI agent

เนื้อหาประกอบด้วยที่มาและวัตถุประสงค์ ทฤษฎีและข้อกำหนดของระบบ สถาปัตยกรรมแบบแบ่งชั้น แบบจำลองข้อมูล การออกแบบ REST API ความปลอดภัย วงจรการเผยแพร่ แบบแผนการออกแบบ ผลการทดสอบ และการประเมินตามหลัก SOLID รวมทั้งข้อจำกัดและข้อเสนอแนะเพื่อพัฒนาต่อ

คณะผู้จัดทำขอขอบพระคุณ รศ. ดร.ปัญญาพล หอระตะ สำหรับคำแนะนำในการเรียนและการพัฒนาโครงงาน และขอขอบคุณสมาชิกทุกคนที่ร่วมพัฒนาระบบและจัดทำเอกสาร หากรายงานมีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้

คณะผู้จัดทำ

สารบัญ

บทคัดย่อ 01

คำนำ 02

บทที่ 1 บทนำ 05

1.1 ที่มาและความสำคัญ 05

1.2 วัตถุประสงค์ 05

1.3 ขอบเขตและข้อจำกัด 06

1.4 วิธีดำเนินงาน 06

1.5 ประโยชน์ที่คาดว่าจะได้รับ 07

บทที่ 2 ทฤษฎีและข้อกำหนดของระบบ 09

2.1 ภาพรวมระบบ 09

2.2 ผู้ใช้งานและความต้องการ 09

2.3 เทคโนโลยีที่ใช้ 09

2.4 เครื่องมือของ AI agent และ Model Context Protocol 10

2.5 ระบบทะเบียนและตลาดกลางซอฟต์แวร์ 10

2.6 สถาปัตยกรรมแบบแบ่งชั้นและ MVC 10

2.7 REST API และ OpenAPI 11

2.8 Spring Boot, JPA และ Spring Security 11

2.9 ฐานข้อมูลเชิงสัมพันธ์และ PostgreSQL 11

2.10 หลักการ SOLID 11

2.11 GoF Behavioral Design Patterns ที่ใช้ 12

2.12 การทดสอบซอฟต์แวร์ 12

2.13 Docker และการส่งมอบอย่างต่อเนื่อง 12

บทที่ 3 การวิเคราะห์และออกแบบระบบ 13

3.1 สถาปัตยกรรมระบบ 13

3.2 Use Case และคำอธิบาย 13

3.3 Domain Model 14

3.4 การออกแบบฐานข้อมูล 14

3.5 REST API และการตรวจข้อมูล 16
3.6 ความปลอดภัยและวงจรการเผยแพร่ 17
3.7 Design Patterns ในโค้ด 18
3.8 Sequence Diagrams 18
3.9 Activity Diagram 19
3.10 Component และ Deployment 20
3.11 การประเมินตามหลัก SOLID 20
3.12 การทดสอบและการนำระบบขึ้นให้บริการ 21
บทที่ 4 สรุปผลและข้อเสนอแนะ 22
4.1 สรุปผลการดำเนินงาน 22
4.2 ข้อจำกัด 22
4.3 ข้อเสนอแนะ 22
4.4 บทสรุป 23
เอกสารอ้างอิง 24
ภาคผนวก ก พจนานุกรมข้อมูล 25

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ปัจจุบันระบบปัญญาประดิษฐ์เชิงตัวแทน (AI agent) ไม่ได้ทำงานเพียงการสนทนา แต่สามารถเรียกใช้เครื่องมือภายนอก เช่น การค้นหาข้อมูล การอ่านไฟล์ การเรียก API และการเชื่อมต่อฐานข้อมูล เพื่อดำเนินงานแทนผู้ใช้ได้ มาตรฐานเปิดอย่าง Model Context Protocol (MCP) ถูกเสนอขึ้นเพื่อให้แอปพลิเคชันที่ใช้โมเดลภาษาเชื่อมต่อกับเครื่องมือและแหล่งข้อมูลภายนอกด้วยรูปแบบเดียวกัน [3] ส่งผลให้มีนักพัฒนาสร้างเครื่องมือปลั๊กอิน และชุดทักษะสำหรับ AI agent เพิ่มมากขึ้น

อย่างไรก็ตาม เครื่องมือเหล่านี้มักกระจายอยู่ตามที่เก็บซอร์สโค้ดและเว็บไซต์ต่าง ๆ ทำให้เกิดปัญหาหลักอย่างน้อยสี่ประการ ได้แก่ (1) ผู้ใช้ค้นหาเครื่องมือที่ตรงกับความต้องการได้ยาก เพราะไม่มีการจัดหมวดหมู่และแท็กที่เป็นมาตรฐานเดียวกัน (2) ไม่มีตัวชี้วัดคุณภาพ เช่น คะแนนและรีวิวจากผู้ใช้จริง (3) ผู้พัฒนาขาดช่องทางจัดการเวอร์ชันและประกาศการเปลี่ยนแปลงอย่างเป็นระบบ และ (4) ไม่มีขั้นตอนตรวจสอบก่อนเผยแพร่ ทำให้ผู้ใช้ไม่มั่นใจในความน่าเชื่อถือของเครื่องมือ ซึ่งเป็นประเด็นสำคัญเมื่อเครื่องมือเหล่านั้นอาจเข้าถึงข้อมูลหรือสั่งการระบบแทนผู้ใช้

ในวงการซอฟต์แวร์ทั่วไป ปัญหาลักษณะนี้ถูกแก้ด้วยระบบทะเบียนและตลาดกลาง (registry/marketplace) เช่น ศูนย์รวมแพ็กเกจของภาษาโปรแกรมและตลาดส่วนขยายของเครื่องมือพัฒนา ซึ่งให้บริการค้นหา จัดการเวอร์ชัน และแสดงความนิยมของผลงาน แต่สำหรับเครื่องมือของ AI agent ยังไม่มีระบบที่รวมความสามารถดังกล่าวพร้อมกระบวนการอนุมัติโดยผู้ดูแลไว้ในที่เดียว

คณะผู้จัดทำจึงพัฒนา PrimeSkill เว็บแอปพลิเคชันที่ให้นักพัฒนาลงทะเบียนและเผยแพร่ข้อมูล (metadata) และเวอร์ชันของเครื่องมือ ให้ผู้ดูแลระบบ (ADMIN) ตรวจสอบและอนุมัติก่อนเผยแพร่ และให้ผู้ใช้ค้นหา กรอง จัดเรียง ให้คะแนน และเขียนรีวิวได้ โครงงานนี้ยังใช้เป็นโอกาสประยุกต์หลักการออกแบบซอฟต์แวร์ ได้แก่ SOLID สถาปัตยกรรมแบบแบ่งชั้น และแบบแผนการออกแบบกลุ่มพฤติกรรม (behavioral design patterns) ของ GoF ในระบบที่ใช้งานได้จริง [11][12]

1.2 วัตถุประสงค์

1) เพื่อออกแบบและพัฒนาเว็บแอปพลิเคชัน PrimeSkill สำหรับลงทะเบียน เผยแพร่ ค้นหา และรีวิวเครื่องมือหรือปลั๊กอินของ AI agent
2) เพื่อพัฒนาระบบบัญชีผู้ใช้ที่รองรับการสมัครสมาชิก เข้าสู่ระบบ จัดการโปรไฟล์ และควบคุมสิทธิ์ผู้ใช้ทั่วไป (USER) กับผู้ดูแลระบบ (ADMIN) อย่างปลอดภัย
3) เพื่อพัฒนาระบบจัดการเครื่องมือและเวอร์ชัน รวมถึงกระบวนการอนุมัติ DRAFT ⟶ PENDING ⟶ PUBLISHED ⟶ DEPRECATED โดยผู้ดูแลระบบ
4) เพื่อพัฒนาระบบค้นหา กรองตามหมวดหมู่และแท็ก จัดเรียง และแบ่งหน้าผลลัพธ์ พร้อมระบบรีวิวและคะแนนเฉลี่ย
5) เพื่อประยุกต์ใช้หลักการ SOLID สถาปัตยกรรมแบบแบ่งชั้น และ GoF Behavioral Patterns (Strategy, Observer, State) ในการออกแบบและพัฒนาระบบ
6) เพื่อทดสอบระบบอย่างเป็นระบบ และนำระบบขึ้นให้บริการบนอินเทอร์เน็ตด้วย Docker, GitHub Actions และบริการคลาวด์

1.3 ขอบเขตและข้อจำกัด

1.3.1 ขอบเขต

1) ระบบเป็นเว็บแอปพลิเคชันแบบ single service พัฒนาด้วย Java 17, Spring Boot, Spring Data JPA, Spring Security และ Thymeleaf ใช้ฐานข้อมูล PostgreSQL (Supabase)
2) ผู้ใช้แบ่งเป็น ผู้เยี่ยมชม (ดูและค้นหาเครื่องมือที่เผยแพร่แล้ว), USER (สมัคร จัดการโปรไฟล์ เผยแพร่เครื่องมือของตน รีวิว) และ ADMIN (อนุมัติ/ปฏิเสธเครื่องมือ จัดการหมวดหมู่และแท็ก)
3) จัดการเครื่องมือและเวอร์ชัน (เพิ่ม ดู แก้ไข ลบ) พร้อมวงจรสถานะ DRAFT ⟶ PENDING ⟶ PUBLISHED ⟶ DEPRECATED และกำหนดหมวดหมู่/แท็กให้เครื่องมือ
4) ค้นหาจากชื่อและคำอธิบาย กรองตามหมวดหมู่และแท็ก จัดเรียงตามใหม่สุด ความนิยม คะแนน และความเกี่ยวข้อง พร้อมแบ่งหน้า
5) รีวิวและให้คะแนน 1–5 ได้หนึ่งรายการต่อผู้ใช้ต่อเครื่องมือ พร้อมแสดงคะแนนเฉลี่ยและจำนวนรีวิว
6) มี REST API ที่มีเอกสาร OpenAPI/Swagger, Dockerfile, Docker Compose, GitHub Actions และการติดตั้งใช้งานสาธารณะ

1.3.2 ข้อจำกัด

1) ไม่มีการเข้าสู่ระบบผ่านบุคคลที่สาม (OAuth) การยืนยันอีเมล หรือการกู้คืนรหัสผ่าน

2)ไม่รองรับการอัปโหลดไฟล์ไบนารีของเครื่องมือเก็บเฉพาะข้อมูลอธิบายและลิงก์ระบบไม่ได้รันหรือตรวจความปลอดภัยของโค้ดเครื่องมือการอนุมัติเป็นการตรวจสอบโดยผู้ดูแลระบบด้วยตนเอง

3) ไม่มีระบบชำระเงิน การค้นหาเชิงความหมายหรือแมชชีนเลิร์นนิง การแชตเรียลไทม์ แอปพลิเคชันมือถือเนทีฟ และมีบทบาทผู้ดูแลเพียงระดับเดียว

4) การแจ้งเตือนจากแบบแผน Observer บันทึกเป็นเหตุการณ์ภายในระบบ (audit/log) ไม่ส่งอีเมลหรือ webhook จริง

5) ระบบใช้บริการระดับฟรีของผู้ให้บริการคลาวด์ จึงมีข้อจำกัดด้านทรัพยากรและจำนวนผู้ใช้พร้อมกัน

1.4 วิธีดำเนินงาน

คณะผู้จัดทำแบ่งงานเป็น 5 บทบาท (Role A–E) ตามโมดูลของระบบ และพัฒนาแบบเป็นรอบสั้น (sprint) โดยแต่ละคนพัฒนาบนสาขาของตนเอง ส่งคำขอรวมโค้ด (pull request) เข้าสาขา develop และต้องผ่านการทบทวนโดยเพื่อนร่วมทีมก่อนรวมเสมอ [2] ขั้นตอนดำเนินงานมีดังนี้

1) ศึกษาและรวบรวมความต้องการ: ศึกษาแนวคิด AI agent, MCP และระบบทะเบียนซอฟต์แวร์ที่มีอยู่ กำหนดผู้ใช้ 3 กลุ่ม (ผู้เยี่ยมชม, USER, ADMIN) และกรณีใช้งานหลัก

2) ออกแบบระบบ: ออกแบบสถาปัตยกรรมแบบแบ่งชั้น แผนภาพ ER สัญญา REST API รูปแบบข้อผิดพลาด และแผนภาพของแบบแผน Strategy, Observer และ State

3) เตรียมสภาพแวดล้อม (Sprint 0): สร้างโครงโปรเจกต์ Spring Boot บน Maven และ Java 17 ตั้งค่าฐานข้อมูล PostgreSQL บน Supabase และบนเครื่องผ่าน Docker Compose ตั้งค่า GitHub Actions และสาขาตามรูปแบบที่กำหนด

4) พัฒนาโมดูลหลัก (Sprint 1–2): พัฒนาตามบทบาทในตารางที่ 1 โดยเริ่มจากระบบบัญชีผู้ใช้และข้อผิดพลาดมาตรฐาน ตามด้วยเครื่องมือและหมวดหมู่ แล้วจึงพัฒนาการค้นหา รีวิว และการเผยแพร่พร้อมกัน

5) พัฒนาหน้าเว็บและรวมระบบ (Sprint 3): สร้างหน้า Thymeleaf ได้แก่ หน้าแรก รายการและรายละเอียดเครื่องมือ สมัคร/เข้าสู่ระบบ โปรไฟล์ แดชบอร์ดเจ้าของเครื่องมือ รีวิวของฉัน และหน้าตรวจสอบของผู้ดูแล รวมโมดูลเข้าด้วยกัน ตรวจ CSRF การตรวจสอบข้อมูล การเข้าถึงด้วยแป้นพิมพ์ และประสิทธิภาพของคำสั่ง SQL

6) ทดสอบ: ทดสอบหน่วย ทดสอบ API และสิทธิ์ ทดสอบบูรณาการกับ PostgreSQL จริง และทดสอบ flow ครบวงจร สมัคร ⟶ เข้าสู่ระบบ ⟶ สร้างเครื่องมือ ⟶ ส่งอนุมัติ ⟶ อนุมัติ ⟶ ค้นหา ⟶ รีวิว

7) นำระบบขึ้นให้บริการ (Sprint 4): สร้าง Docker image แบบหลายขั้นตอน ตั้งค่าตัวแปรสภาพแวดล้อมและความลับ ติดตั้งบน Vercel เชื่อมฐานข้อมูล Supabase และทดสอบหลังติดตั้ง

8) สรุปผล เขียนรายงาน และนำเสนอผลงาน: จัดทำเอกสารประกอบ (พจนานุกรมข้อมูล แผนภาพ) รายงานผลการทดสอบ และสาธิตระบบ

ตารางที่ 1 การแบ่งบทบาทและโมดูลของระบบ

บทบาท	ผู้รับผิดชอบ	โมดูล	ตารางข้อมูล
A	ณัฐกรณ์ อินธิสาร	บัญชีผู้ใช้ เข้าสู่ระบบ/ออกจากระบบ โปรไฟล์ session CSRF สิทธิ์ USER/ADMIN	users, user_profiles
B	ณัชพล เพ็งพล	สร้าง/ดู/แก้ไข/ลบเครื่องมือ การตรวจสอบข้อมูล สิทธิ์เจ้าของ หมวดหมู่	tools, categories
C	ศุภกร กรมรินทร์	ค้นหา กรองตามหมวดหมู่และแท็ก จัดเรียง แบ่งหน้า จัดการแท็ก (Strategy)	tags, tool_tags
D	ณภัทร อรัญพูล	สร้าง/แก้ไข/ลบรีวิว รีวิวของฉัน คะแนนเฉลี่ย (Observer)	reviews
E	ธนินธร อันทรบุตร	เวอร์ชัน การเผยแพร่และอนุมัติ (State) การรวมระบบ CI Docker Deployment	tool_versions

1.5 ประโยชน์ที่คาดว่าจะได้รับ

(1)ได้ระบบทะเบียนกลางที่ช่วยให้ผู้ใช้ค้นหา เปรียบเทียบ และเลือกใช้เครื่องมือของ AI agent ได้สะดวกขึ้น

(2)ผู้พัฒนามีช่องทางเผยแพร่ จัดการเวอร์ชัน และรับข้อเสนอแนะจากผู้ใช้

(3)กระบวนการอนุมัติโดยผู้ดูแลช่วยเพิ่มความน่าเชื่อถือของเครื่องมือที่แสดงในระบบ

(4)ได้ตัวอย่างการประยุกต์ SOLID สถาปัตยกรรมแบบแบ่งชั้น และ GoF Behavioral Patterns ในระบบจริง พร้อมเอกสารและชุดทดสอบ ใช้เป็นแนวทางศึกษาต่อได้

(5)สมาชิกในทีมได้ฝึกการพัฒนาซอฟต์แวร์เป็นทีม ทั้งการจัดการสาขา การทบทวนโค้ด การรวมระบบ การทดสอบ และการติดตั้งใช้งานจริง

(6)ต่อยอดในอนาคตได้ เช่นเพิ่มOAuthการตรวจสอบความปลอดภัยของเครื่องมืออัตโนมัติหรือการเชื่อมต่อกับมาตรฐาน MCP โดยตรง

บทที่ 2 ทฤษฎีและข้อกำหนดของระบบ

2.1 ภาพรวมระบบ

PrimeSkill เป็นเว็บแอปพลิเคชันทะเบียนและตลาดกลางสำหรับเครื่องมือและปลั๊กอินของ AI agent ผู้เข้าชมดูรายการที่เผยแพร่ได้โดยไม่ต้องเข้าสู่ระบบ ส่วนการแก้ข้อมูล รีวิว และการดูแลระบบต้องใช้บัญชีตามสิทธิ์ แอปมีหน้าเว็บ Thymeleaf และ REST API ซึ่งใช้ Service ร่วมกันเพื่อคงกฎธุรกิจชุดเดียว

2.2 ผู้ใช้งานและความต้องการ

ตารางที่ 2 ผู้ใช้งานและความต้องการหลัก

ผู้ใช้งาน	ความต้องการหลัก	เงื่อนไข
ผู้เยี่ยมชม	ค้นหา ดูรายละเอียด สมัครสมาชิก	เห็นเฉพาะรายการที่เผยแพร่
USER	จัดการโปรไฟล์ รีวิว และเครื่องมือของตน	ตรวจสิทธิ์และ ownership
เจ้าของเครื่องมือ	แก้ข้อมูล/เวอร์ชัน และส่งตรวจ	แก้เวอร์ชันและแท็กได้เมื่อ DRAFT
ADMIN	จัดการหมวดหมู่/แท็ก และกลั่นกรอง	เปลี่ยนสถานะตาม transition ที่อนุญาต

2.2.1 ข้อกำหนดเชิงหน้าที่

(1)ค้นหาเครื่องมือตามคำค้น หมวดหมู่ แท็ก ลำดับ และหน้า

(2)ดูรายละเอียด คะแนนเฉลี่ย จำนวนรีวิว และเวอร์ชันของรายการที่เผยแพร่

(3)ให้เจ้าของจัดการเครื่องมือของตนและส่งรายการเข้าสู่คิวอนุมัติ

(4)ให้ผู้ใช้จัดการรีวิวของตน โดยจำกัดหนึ่งรีวิวต่อผู้ใช้ต่อเครื่องมือ

(5)ให้ผู้ดูแลจัดการข้อมูลอ้างอิงและอนุมัติหรือปฏิเสธการเผยแพร่

2.2.2 คุณภาพและความปลอดภัย

ระบบต้องแยกชั้นงานเพื่อให้ทดสอบและดูแลได้ ตรวจ validation ที่ขอบเขต request ป้องกันการแก้ข้อมูลข้ามเจ้าของ ใช้ session authentication และ CSRF สำหรับคำขอเขียนข้อมูล บังคับ referential integrity ด้วย foreign key/unique/check constraints และบันทึก audit หลัง transaction สำเร็จ ข้อกำหนดนี้สอดคล้องกับใบงานเรื่อง Layered Architecture, REST API, SQL database และการทดสอบ [1]

2.3 เทคโนโลยีที่ใช้

ตารางที่ 3 เทคโนโลยีและบทบาทในระบบ

ส่วน	เทคโนโลยี	บทบาท
ภาษา/runtime	Java 17, Spring Boot 4.1.1, Maven	เริ่มแอปและจัดการ dependencies
Web/API	Spring MVC, Thymeleaf, REST, OpenAPI (springdoc)	รับคำขอ แสดงเว็บ และกำหนด API contract
Persistence	Spring Data JPA, Hibernate, PostgreSQL (Supabase)	จัดการ entity และข้อมูล
Security	Spring Security, session, BCrypt, CSRF	ยืนยันตัวตนและตรวจสิทธิ์
ทดสอบ/ส่งมอบ	JUnit 5, Mockito, MockMvc, PostgreSQL integration, Docker, GitHub Actions, Vercel	ทดสอบและจัดส่งตาม configuration

2.4 เครื่องมือของ AI agent และ Model Context Protocol

AI agent คือระบบที่ใช้โมเดลภาษาเป็นแกนกลางในการวางแผนและเรียกใช้เครื่องมือภายนอกเพื่อทำงานให้สำเร็จ Model Context Protocol (MCP) เป็นโปรโตคอลเปิดที่กำหนดรูปแบบการเชื่อมต่อระหว่างแอปพลิเคชัน AI กับเครื่องมือและแหล่งข้อมูล ทำให้เครื่องมือชิ้นเดียวนำไปใช้ได้กับหลายแอปพลิเคชัน [3] การมีมาตรฐานกลางเช่นนี้ทำให้เกิดความต้องการระบบทะเบียนกลางที่รวบรวมข้อมูลเครื่องมือ

2.5 ระบบทะเบียนและตลาดกลางซอฟต์แวร์

ระบบทะเบียนซอฟต์แวร์ เช่น ศูนย์รวมแพ็กเกจและตลาดส่วนขยาย มีองค์ประกอบร่วมกันคือ ข้อมูลผลงานพร้อมคำอธิบาย หมวดหมู่และแท็ก การกำหนดเวอร์ชัน การค้นหาและจัดอันดับ และข้อมูลจากชุมชน เช่น คะแนนและรีวิว PrimeSkill ศึกษาองค์ประกอบเหล่านี้เป็นแนวทางเชิงแนวคิด แล้วออกแบบและพัฒนาขึ้นใหม่เองทั้งหมดให้เหมาะกับเครื่องมือของ AI agent พร้อมเพิ่มกระบวนการอนุมัติโดยผู้ดูแลระบบก่อนเผยแพร่

2.6 สถาปัตยกรรมแบบแบ่งชั้นและ MVC

สถาปัตยกรรมแบบแบ่งชั้น (layered architecture) แยกความรับผิดชอบของระบบเป็นชั้น เช่น การนำเสนอ (presentation) ตรรกะธุรกิจ (service) การเข้าถึงข้อมูล (repository) และโดเมน ช่วยให้แก้ไข ทดสอบ และขยายระบบได้ง่าย [4] PrimeSkill ใช้ Spring MVC ให้ controller ทำหน้าที่รับส่งข้อมูล HTTP, service รับผิดชอบกฎทางธุรกิจ สิทธิ์ และธุรกรรม และ repository รับผิดชอบการอ่านเขียนฐานข้อมูล โดยไม่ให้ controller เรียก repository โดยตรง [1]

ตารางที่ 4 รูปแบบเชิงสถาปัตยกรรมที่ใช้ในระบบ

รูปแบบ	ตัวอย่างในระบบ	ประโยชน์
MVC	Web Controller, model/DTO, Thymeleaf template	แยก request, data และ view
Repository	Spring Data JPA interfaces	รวม query และการเข้าถึงข้อมูล
Service Layer	service interfaces และ implementations	รวม use case, business rule, transaction
DTO + Mapper	request/response DTO และ mapper classes	ไม่เปิด entity เป็น API contract
Dependency Injection	constructor injection ผ่าน Spring	เปลี่ยน collaborator และทดสอบได้

2.7 REST API และ OpenAPI

REST เป็นรูปแบบสถาปัตยกรรมสำหรับระบบแบบกระจายบนเว็บ ที่ใช้ทรัพยากรที่ระบุด้วย URL และกริยา HTTP ที่มีความหมายชัดเจน [5] ระบบกำหนดรหัสสถานะ HTTP และรูปแบบข้อผิดพลาด JSON ให้สม่ำเสมอ และใช้ OpenAPI Specification ร่วมกับ springdoc เพื่อสร้างเอกสาร API และหน้า Swagger UI อัตโนมัติ [6]

2.8 Spring Boot, JPA และ Spring Security

Spring Boot ช่วยตั้งค่าแอปพลิเคชัน Java ให้พร้อมใช้งานอย่างรวดเร็ว **[**7] Spring Data JPA ใช้แมปวัตถุ (entity**)** กับตารางฐานข้อมูลเชิงสัมพันธ์และลด boilerplate ในชั้น data access [8] และ Thymeleaf ใช้สร้างหน้าเว็บฝั่งเซิร์ฟเวอร์ ส่วน Spring Security จัดการการพิสูจน์ตัวตนแบบ session การเข้ารหัสรหัสผ่านด้วย BCrypt การป้องกัน CSRF และการควบคุมสิทธิ์ตามบทบาท ซึ่งสอดคล้องกับแนวปฏิบัติด้านความปลอดภัยของเว็บแอปพลิเคชันของ OWASP [9]

2.9 ฐานข้อมูลเชิงสัมพันธ์และ PostgreSQL

PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ที่รองรับธุรกรรม ข้อจำกัดความถูกต้องของข้อมูล (primary key, foreign key, unique, check) และดัชนี โดย foreign key รักษา referential integrity และ unique constraint บังคับค่าหรือชุดค่าไม่ซ้ำ **[10]** โครงงานนี้ใช้ออกแบบความสัมพันธ์แบบหนึ่งต่อหนึ่ง หนึ่งต่อกลุ่ม และกลุ่มต่อกลุ่ม พร้อมใช้ข้อจำกัดในฐานข้อมูลเป็นด่านสุดท้ายป้องกันข้อมูลซ้ำหรือไม่ถูกต้อง

2.10 หลักการ SOLID

SOLID ประกอบด้วย Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation และ Dependency Inversion [11] ซึ่งช่วยให้โค้ดแยกหน้าที่ชัดเจน ขยายความสามารถได้โดยไม่

แก้โค้ดเดิม และพึ่งพา interface มากกว่าคลาสจริง โครงงานนี้ใช้ constructor injection และกำหนด interface ตามกรณีใช้งานจริง

2.11 GoF Behavioral Design Patterns ที่ใช้

ตารางที่ 5 แบบแผนการออกแบบที่ใช้ในโครงงาน

แบบแผน	ปัญหาที่แก้	การใช้ใน PrimeSkill
Strategy	ต้องการสลับวิธีจัดเรียงโดยไม่แก้โค้ดค้นหา	ToolSortStrategy พร้อมคลาส Newest, Popularity, Rating และ Relevance โดย ToolSearchService เลือกตาม ToolSortOption
Observer	ต้องการแจ้งเหตุการณ์โดยไม่ให้ตรรกะหลักผูกกับผู้รับ	ReviewService ส่ง ReviewCreatedEvent หลังบันทึกรีวิว และ ReviewAuditListener รับเหตุการณ์หลังธุรกรรมสำเร็จ
State	พฤติกรรมขึ้นกับสถานะของเครื่องมือ	PublishingState และ PublishingStateMachine ควบคุมการเปลี่ยนสถานะ DRAFT, PENDING, PUBLISHED, DEPRECATED

ทั้งสามแบบแผนอยู่ในกลุ่มพฤติกรรมตามแคตตาล็อกของ GoF [12]

2.12 การทดสอบซอฟต์แวร์

โครงงานใช้ JUnit 5 และ Mockito สำหรับการทดสอบหน่วย (unit test) ของ service และ mapper ใช้ Spring Boot Test กับ MockMvc ทดสอบ controller การตรวจสอบข้อมูลและสิทธิ์ และใช้การทดสอบเชิงบูรณาการกับ PostgreSQL จริงเพื่อตรวจข้อจำกัดของฐานข้อมูลและกรณีการทำงานพร้อมกัน (concurrency) เช่น การกดอนุมัติซ้อนกันหรือการเขียนรีวิวซ้ำ ซึ่งการทดสอบด้วยฐานข้อมูลจำลองอย่างเดียวอาจตรวจไม่พบ

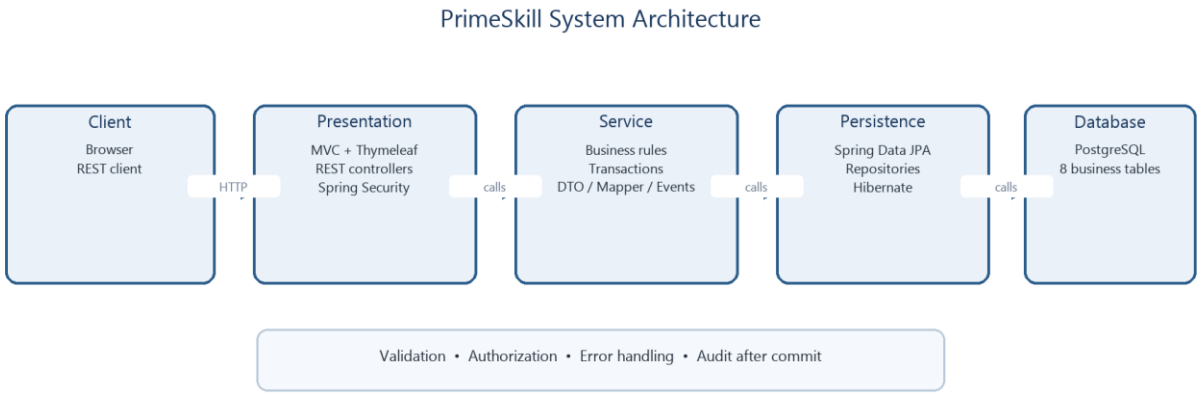
2.13 Docker และการส่งมอบอย่างต่อเนื่อง

Docker ใช้บรรจุแอปพลิเคชันพร้อมสภาพแวดล้อมให้ทำงานเหมือนกันทุกเครื่อง ส่วน GitHub Actions ใช้สร้างและทดสอบโค้ดอัตโนมัติทุกครั้งที่มีการส่งโค้ด เพื่อให้มั่นใจว่าโค้ดที่รวมเข้าสาขาหลักผ่านการทดสอบแล้ว [13]

บทที่ 3 การวิเคราะห์และออกแบบระบบ

3.1 สถาปัตยกรรมระบบ

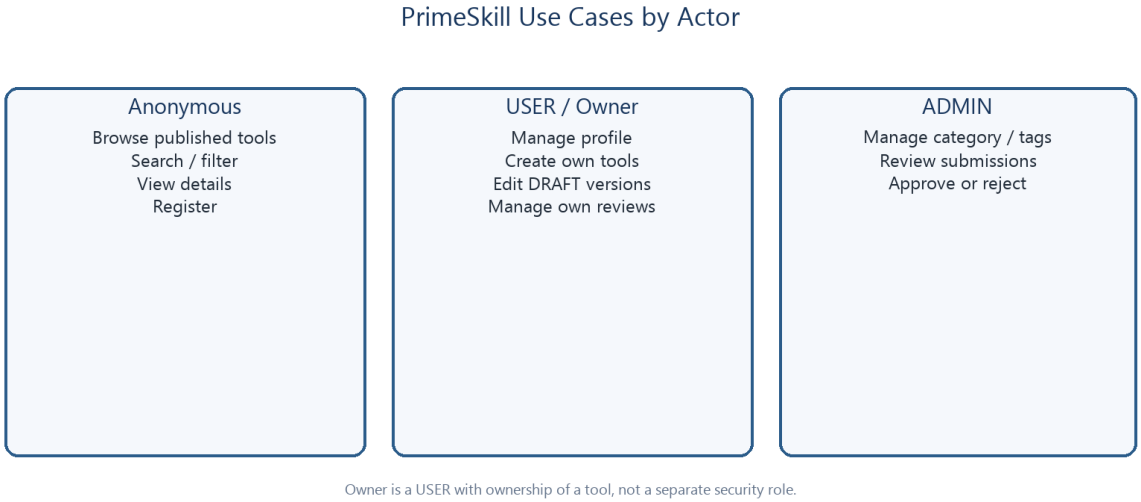
ระบบแบ่งเป็นชั้น Presentation (REST controller และ Thymeleaf controller) ⟶ Service ⟶ Repository (Spring Data JPA) ⟶ Domain (entity และ enum) ⟶ PostgreSQL โดยมีส่วนข้ามชั้น ได้แก่ config, security, DTO/mapper, exception และ event ระบบไม่ส่ง entity ออกสู่ภายนอก แต่ใช้ DTO แยกสำหรับคำขอและคำตอบ และ Hibernate ตั้งเป็นโหมดตรวจสอบโครงสร้างตาราง (validate) ไม่สร้างตารางเอง [2]



ภาพที่ 1 ภาพรวมสถาปัตยกรรม PrimeSkill

การแบ่งชั้นลดการผูก UI กับ persistence แต่ต้องระวังไม่ให้นโยบายการค้นหา (query policy) ไหลกลับไปอยู่ใน service ซึ่งประเมินไว้ในหัวข้อ 3.11

3.2 Use Case และคำอธิบาย

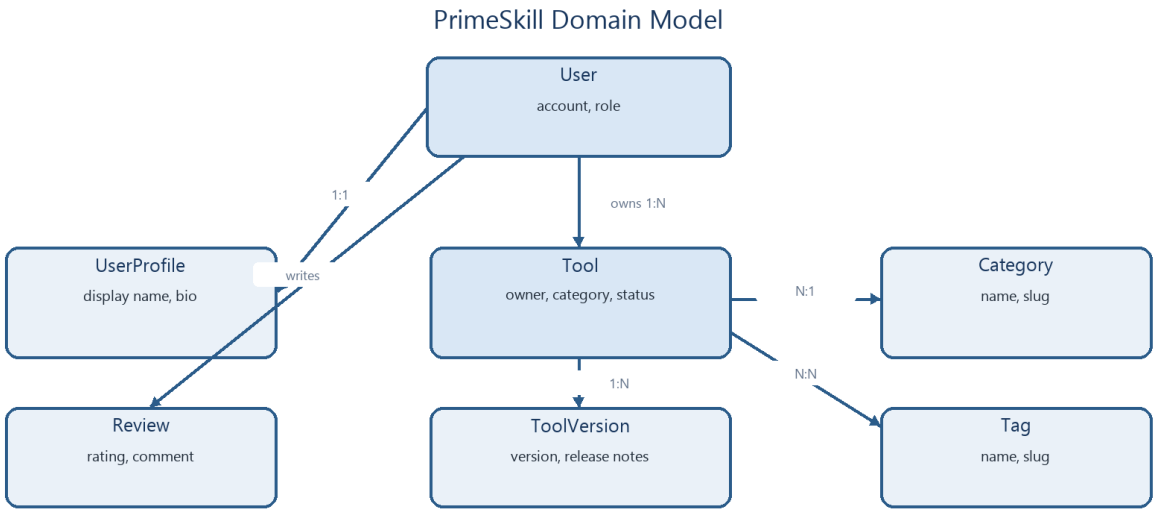


ภาพที่ 2 กลุ่ม Use Case แยกตามผู้เยี่ยมชม ผู้ใช้/เจ้าของ และผู้ดูแล

ตารางที่ 6 คำอธิบาย Use Case

Use Case	ผู้เริ่มต้น	ผลลัพธ์/เงื่อนไข
ค้นหาและเรียกดู	ผู้เยี่ยมชม/USER	q, category, tags, sort, page, size; แสดงรายการที่เผยแพร่
จัดการเครื่องมือของตน	USER/เจ้าของ	ตรวจ ownership; แก้ข้อมูลตามสถานะ
ส่งตรวจ/พิจารณาเผยแพร่	เจ้าของ + ADMIN	DRAFT⟶PENDING; approve เป็น PUBLISHED หรือ reject กลับ DRAFT
จัดการรีวิว	USER	ตรวจสถานะ; หนึ่งรีวิวต่อผู้ใช้ต่อเครื่องมือ
จัดการหมวดหมู่/แท็ก	ADMIN	แก้ข้อมูลอ้างอิงสำหรับจัดหมวดและค้นหา

3.3 Domain Model



ภาพที่ 3 แบบจำลองแนวคิดของข้อมูลหลักและความสัมพันธ์

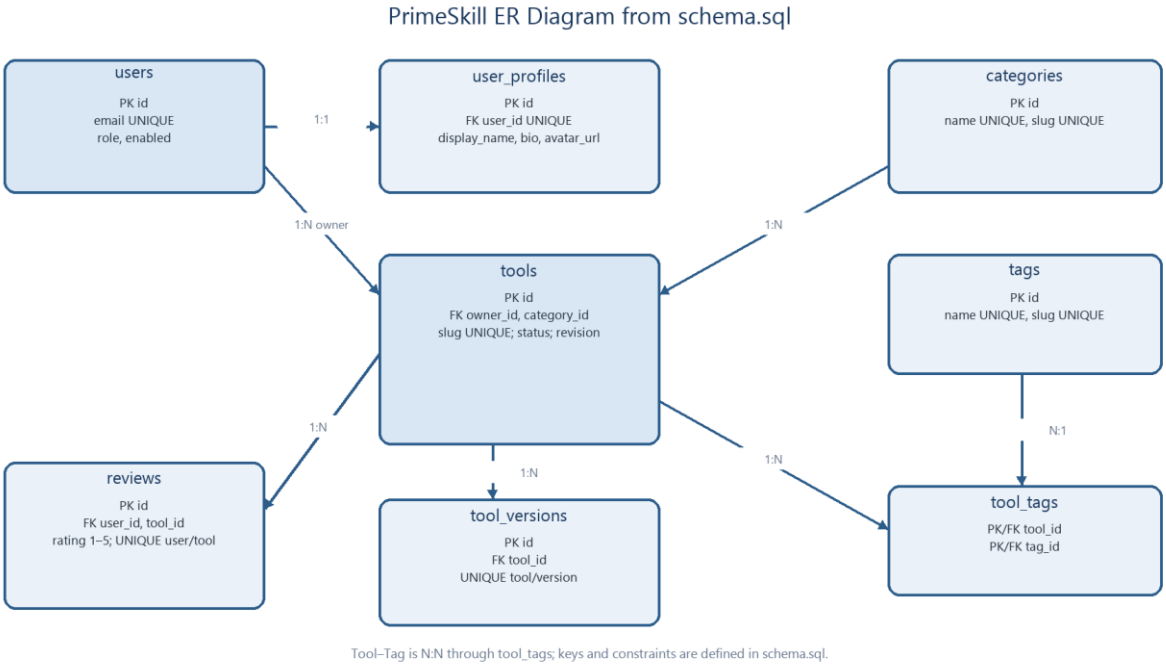
User มีโปรไฟล์หนึ่งชุดและเป็นเจ้าของเครื่องมือได้หลายรายการ เครื่องมือแต่ละรายการอยู่ในหมวดหมู่หนึ่ง มีรีวิวและเวอร์ชันหลายรายการ และเชื่อมโยงแท็กได้หลายค่า ความสัมพันธ์เครื่องมือกับแท็กแยกผ่าน ToolTag เพื่อบังคับคีย์และอ้างอิงได้

3.4 การออกแบบฐานข้อมูล

ฐานข้อมูลมีตารางธุรกิจ 8 ตาราง ได้แก่ users, user_profiles, categories, tools, tags, tool_tags, reviews และ tool_versions รองรับความสัมพันธ์แบบ one-to-one, one-to-many และ many-to-many ตามใบงาน [1][2] ข้อมูลและข้อจำกัดสำคัญของแต่ละตารางสรุปในตารางที่ 7 และรายละเอียดทุกคอลัมน์อยู่ในภาคผนวก ก

ตารางที่ 7 ตารางข้อมูลหลักของระบบ (8 ตาราง)

ตาราง	ข้อมูลและข้อจำกัดสำคัญ
users	email ไม่ซ้ำ, password_hash (BCrypt), role (USER/ADMIN), enabled
user_profiles	ความสัมพันธ์ 1:1 กับ users, display_name, bio, avatar_url
categories	name และ slug ไม่ซ้ำ; ห้ามลบหมวดหมู่ที่ยังมีเครื่องมืออ้างอิง
tools	name, slug ไม่ซ้ำ, คำอธิบาย, repository_url, status, view_count, FK ไปยังเจ้าของและหมวดหมู่
tags / tool_tags	แท็กไม่ซ้ำ; ตารางเชื่อมความสัมพันธ์ N:M ระหว่างเครื่องมือกับแท็ก
reviews	rating ต้องอยู่ระหว่าง 1–5 (CHECK), comment ไม่เกิน 2,000 ตัวอักษร, ไม่ซ้ำต่อ (user_id, tool_id)
tool_versions	version, release_notes, ไม่ซ้ำต่อ (tool_id, version)



ภาพที่ 4 แผนภาพความสัมพันธ์ของข้อมูล (ER Diagram) ของ PrimeSkill

ฐานข้อมูลใช้ primary key ระบุแถว foreign key รักษาความสัมพันธ์ และ unique/check constraints สำหรับ email, slug, คะแนน 1–5, รีวิวซ้ำ และความยาว comment ไม่เกิน 2,000 ตัวอักษร ทำให้ความถูกต้องของข้อมูลไม่ขึ้นกับโค้ดแอปพลิเคชันเพียงอย่างเดียว [**10**]

3.4.1 ความสัมพันธ์และข้อกำหนดสำคัญ

ตารางที่ 8 ความสัมพันธ์ระหว่างตาราง

ความสัมพันธ์	ชนิด	ข้อกำหนด
users — user_profiles	1:1	user_id เป็น FK และ UNIQUE; ลบบัญชีแล้วลบ profile
categories — tools	1:N	category_id เป็น FK; ลบหมวดที่ถูกใช้อยู่ไม่ได้
users — tools	1:N	owner_id ระบุเจ้าของ; service ตรวจ ownership
users/tools — reviews	1:N ต่อด้าน	FK สองด้าน; UNIQUE(user_id, tool_id)
tools — tool_versions	1:N	UNIQUE(tool_id, version); ลบ tool แล้วลบ version
tools — tags	N:N	ผ่าน tool_tags ที่มี composite PK และ FK สองด้าน

3.5 REST API และการตรวจข้อมูล

API ใช้ base path /api/v1 และจัด endpoint ตาม resource เช่น tools, versions, reviews, categories, tags, profile และ authentication เส้นทางค้นหารองรับคำค้น หมวดหมู่ แท็ก การเรียงลำดับ และ pagination ส่วนการเขียนข้อมูลต้องผ่าน session, CSRF และสิทธิ์ที่ถูกต้อง ระบบใช้รหัสสถานะ 200/201/204/400/401/403/404/409/500 และรูปแบบข้อผิดพลาด JSON เดียวกัน พร้อมเอกสาร Swagger UI [2][6]

ตารางที่ 9 กลุ่ม API และการควบคุม

กลุ่ม API	การทำงาน	การควบคุม
Tools	ค้นหา /api/v1/tools; ดูรายละเอียด id หรือ slug	public view แสดงรายการที่เผยแพร่
Versions	อ่านและจัดการเวอร์ชัน	ตรวจ owner; แก้ได้เฉพาะ DRAFT
Reviews	อ่าน/สร้าง/แก้/ลบรีวิว	คะแนน 1–5; ห้าม user รีวิว tool เดิมซ้ำ
Categories/Tags	อ่านข้อมูลและจัดการโดย ADMIN	กำหนดสิทธิ์; name/slug ไม่ซ้ำ
Profile/Auth	สมัคร เข้าสู่ระบบ ปรับ profile	session security และ CSRF

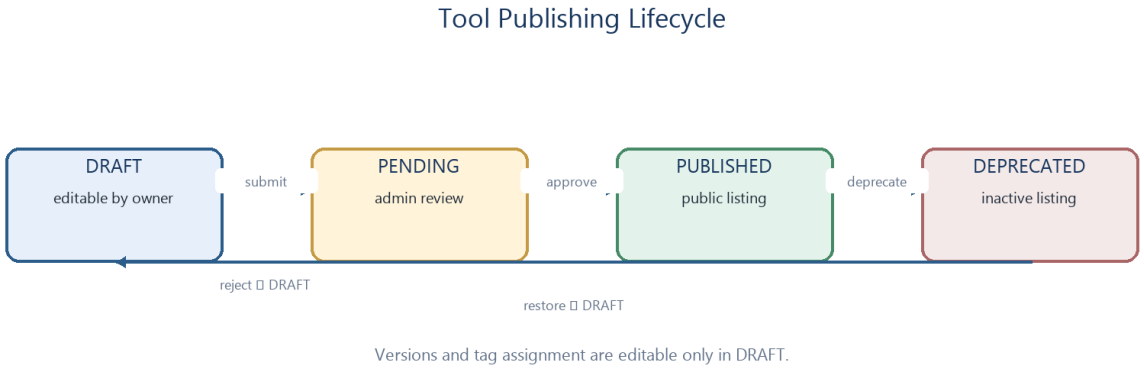
3.6 ความปลอดภัยและวงจรการเผยแพร่

บัญชีใหม่เริ่มต้นเป็น USER ส่วน ADMIN ถูกกำหนดโดยผู้มีสิทธิ์ Owner เป็นความสัมพันธ์ระหว่าง User กับ Tool ไม่ใช่ role แยก การอนุญาตจึงพิจารณาทั้ง role และเจ้าของรายการ การตรวจสิทธิ์เกิดที่ security boundary และ business operation มาตรการหลักมีดังนี้

(1)เก็บรหัสผ่านด้วย BCrypt; ใช้ session ที่ตั้งค่า HttpOnly และ SameSite=Lax; เปิด CSRF สำหรับการแก้ไขข้อมูลผ่านเบราว์เซอร์

(2)ไม่รับตัวตนผู้ใช้ บทบาท หรือเจ้าของจากคำขอ แต่ดึงจากผู้ใช้ที่เข้าสู่ระบบ เพื่อป้องกันการแก้ไขข้อมูลของผู้อื่น

(3)ตรวจสอบข้อมูลด้วย Bean Validation และจำกัดค่า pagination และตัวเลือกการจัดเรียงเฉพาะที่อนุญาต



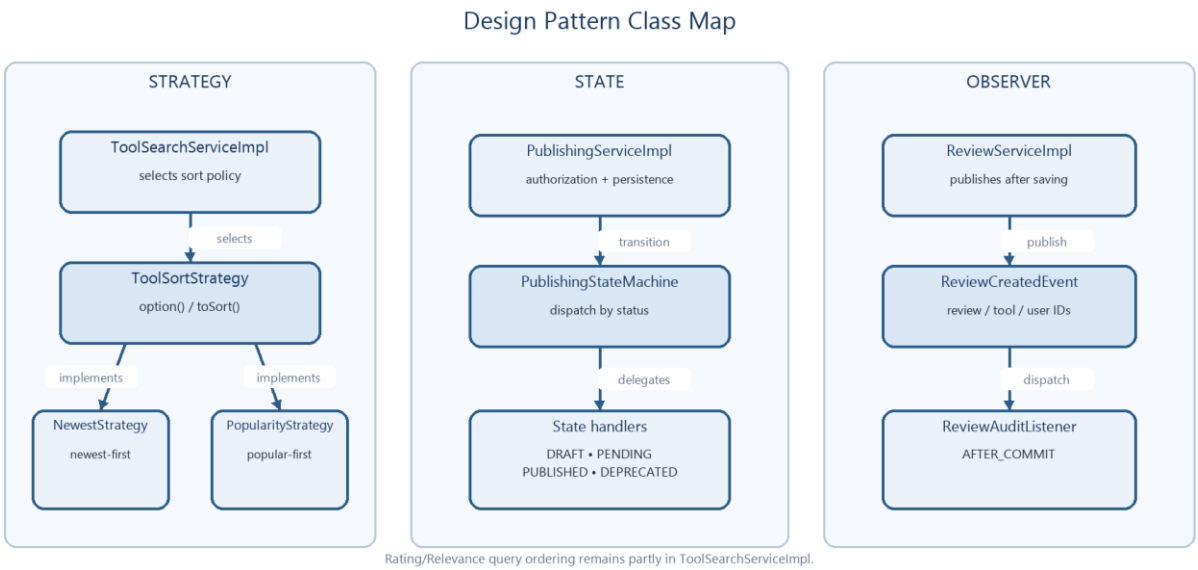
ภาพที่ 5 วงจรสถานะเครื่องมือที่รองรับในระบบ

PublishingStateMachine *แยกการเปลี่ยนสถานะตาม* handler: DRAFT *ส่งตรวจเป็น* PENDING, *ผู้ดูแลอนุมัติเป็น* PUBLISHED *หรือปฏิเสธกลับ* DRAFT, *รายการที่เผยแพร่แล้วเปลี่ยนเป็น* DEPRECATED *และกู้คืนกลับ* DRAFT *ได้ การเปลี่ยนสถานะที่ไม่ถูกต้องถูกปฏิเสธด้วย* domain exception *และตอบกลับเป็น* HTTP 409

ตารางที่ 10 การเปลี่ยนสถานะของเครื่องมือ

สถานะปัจจุบัน	การกระทำ / ผู้กระทำ	สถานะถัดไป	กรณีไม่ถูกต้อง
DRAFT	ส่งอนุมัติ / เจ้าของ	PENDING	HTTP 409
PENDING	อนุมัติ / ADMIN	PUBLISHED	HTTP 409
PENDING	ปฏิเสธ / ADMIN	DRAFT	HTTP 409
PUBLISHED	เลิกใช้ / เจ้าของหรือ ADMIN	DEPRECATED	HTTP 409
DEPRECATED	กู้คืน / เจ้าของ	DRAFT	HTTP 409

3.7 Design Patterns ในโค้ด



ภาพที่ 6 Strategy, State และ Observer พร้อมคลาสหลักที่พบในโค้ด

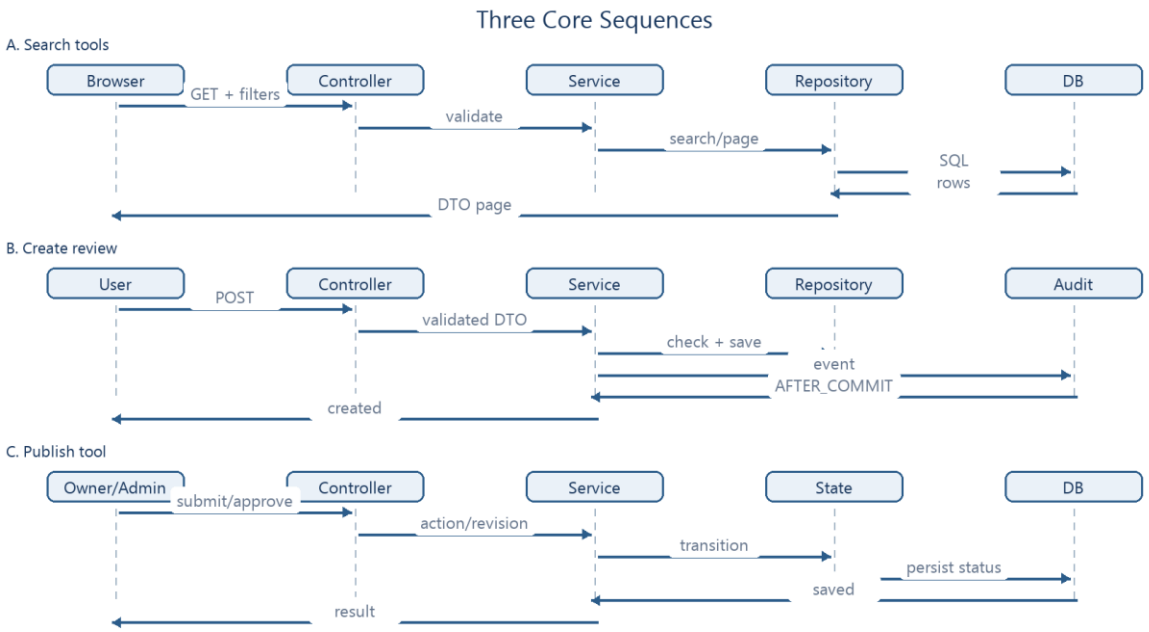
ตารางที่ 11 การใช้แบบแผนการออกแบบและข้อจำกัด

Pattern	ปัญหาที่แก้	การใช้และข้อจำกัด
Strategy	สลับ policy การเรียงลำดับ	Newest/Popularity ใช้ ToolSortStrategy; Rating/Relevance ยังมี query ordering ใน service/repository
State	รวมกฎเปลี่ยนสถานะ	PublishingStateMachine เลือก handler ตาม DRAFT/PENDING/PUBLISHED/DEPRECATED
Observer	แยก audit ออกจากการบันทึกรีวิว	ReviewServiceImpl publish ReviewCreatedEvent; listener ทำงาน AFTER_COMMIT

รูปแบบเชิงสถาปัตยกรรมที่ใช้ร่วมกันคือ Layered Architecture, MVC, Repository, Service Layer, DTO/Mapper และ Dependency Injection แบบแผน Strategy, State และ Observer แต่ละแบบมีปัญหาจริงรองรับ ไม่ได้นำ Spring singleton หรือ Builder มาอ้างเพียงเพื่อให้ครบรายการ [2][12]

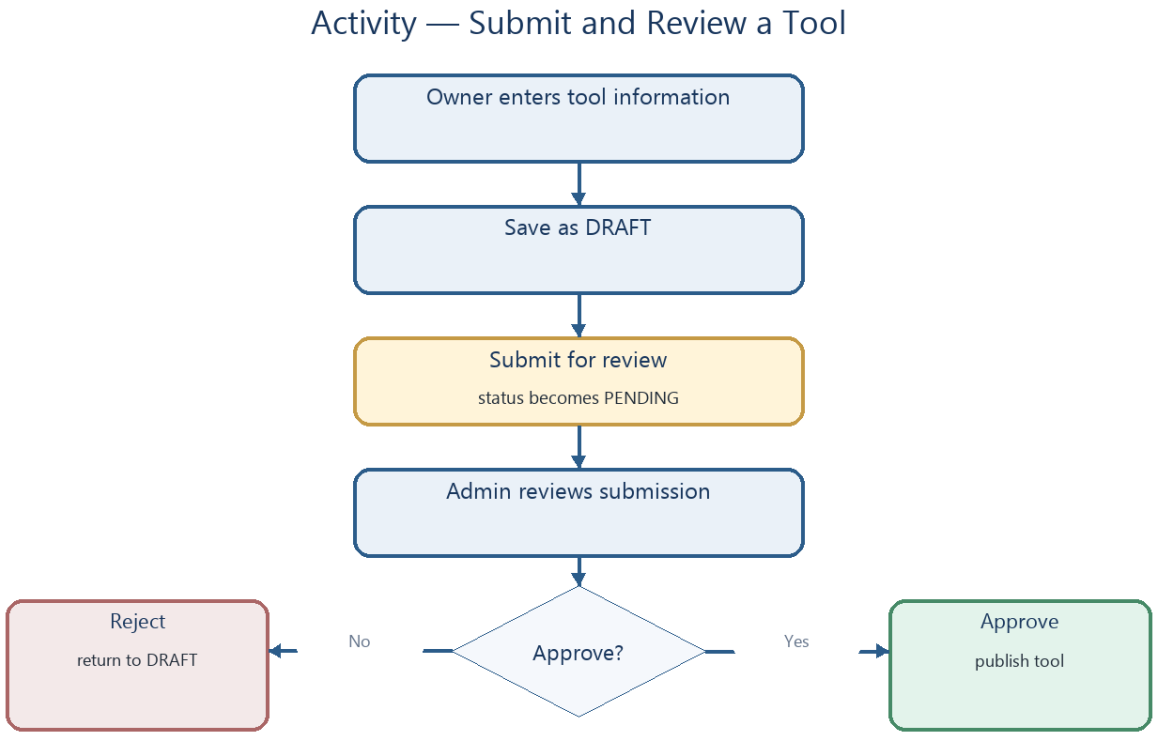
3.8 Sequence Diagrams

แผนภาพต่อไปนี้สรุปเส้นทางจากผู้ใช้ผ่าน Controller และ Service ไปยังฐานข้อมูลของสามสถานการณ์หลัก ได้แก่ การค้นหา การสร้างรีวิว และการเผยแพร่เครื่องมือโดยย่อรายละเอียดภายในเพื่อให้เห็นการตรวจข้อมูลและสิทธิ์



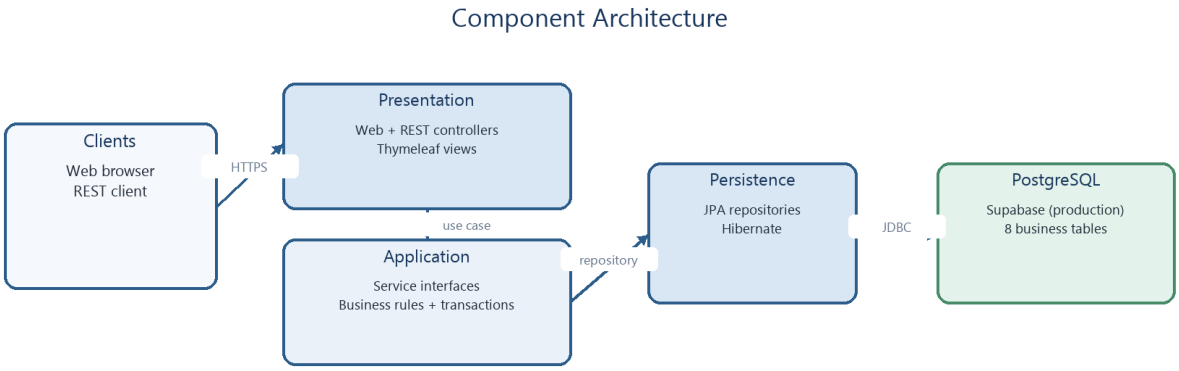
ภาพที่ 7 Sequence Diagrams: ค้นหา สร้างรีวิว และเผยแพร่เครื่องมือ

3.9 Activity Diagram

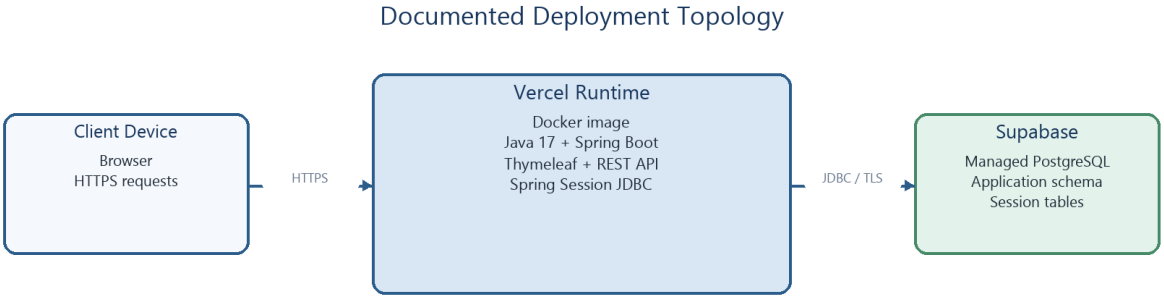


ภาพที่ 8 Activity Diagram: บันทึกร่าง ส่งตรวจ และพิจารณาเผยแพร่

3.10 Component และ Deployment



ภาพที่ 9 Component Diagram ของ controller, service, persistence และ database



ภาพที่ 10 Deployment Diagram ของ PrimeSkill

ระบบบรรจุเป็น Docker image ติดตั้งบน Vercel และเชื่อมต่อ PostgreSQL ที่จัดการโดย Supabase ผ่าน JDBC/TLS โดยมี Dockerfile และ GitHub Actions สำหรับสร้างและทดสอบอัตโนมัติ [2]

3.11 การประเมินตามหลัก SOLID

ประเมินจากโครงสร้างจริงในโค้ดและเอกสารวิเคราะห์ของ repository การมี interface หรือ unit test เพียงอย่างเดียวไม่เพียงพอที่จะรับรองหลักการทั้งหมด [2][11]

ตารางที่ 12 ผลการประเมินตามหลัก SOLID

หลักการ	หลักฐาน	ผลประเมิน
S — SRP	แยก Controller, Service, Repository, Mapper	แบ่งหน้าที่หลักแล้ว; ToolVersionServiceImpl ยัง map/normalize บางส่วน
O — OCP	ToolSortStrategy และ PublishingState	ยังไม่ครบ; rating/relevance query บางส่วนยังเป็นเงื่อนไขใน service
L — LSP	Strategy/state ใช้สัญญาร่วมและ domain exception	ไม่พบ UnsupportedOperationException ใน production Java ตามเอกสาร แต่ยังไม่มี contract tests ครบ

หลักการ	หลักฐาน	ผลประเมิน
I — ISP	interface แยกตาม use case เช่น search/review summary/profile	แนวโน้มดี; ควรตรวจ consumer ของ ToolService/TagService ต่อ
D — DIP	Controller พึ่ง service interface; service พึ่ง repository interface ผ่าน constructor	ยังไม่ครบ interface-only เพราะ mapper/state machine บางตัวเป็น concrete dependency

3.12 การทดสอบและการนำระบบขึ้นให้บริการ

การทดสอบแบ่งเป็นสี่ระดับตามวิธีดำเนินงาน ได้แก่ การทดสอบหน่วยของ service และ mapper (JUnit 5, Mockito) การทดสอบ controller การตรวจข้อมูลและสิทธิ์ (MockMvc) การทดสอบเชิงบูรณาการกับ PostgreSQL จริง และการทดสอบ flow ครบวงจร สมัคร ⟶ เข้าสู่ระบบ ⟶ สร้างเครื่องมือ ⟶ ส่งอนุมัติ ⟶ อนุมัติ ⟶ ค้นหา ⟶ รีวิว

README ของ repository บันทึกผลก่อน merge #13 ที่ SHA 2af3802 ว่ามี Surefire 387 รายการและ Failsafe 349 รายการ รวม 736 Java tests ผ่านโดยไม่มี failure/error/skip ทั้งนี้ตัวเลขดังกล่าวเป็นผลของ commit นั้น ไม่รับรองการเปลี่ยนแปลงภายหลัง และการทดสอบยืนยันพฤติกรรมที่ครอบคลุมแต่ไม่พิสูจน์ SOLID โดยตัวเอง [2]

บทที่ 4 สรุปผลและข้อเสนอแนะ

4.1 สรุปผลการดำเนินงาน

PrimeSkill เป็นเว็บแอปพลิเคชันทะเบียนและตลาดกลางสำหรับเครื่องมือและปลั๊กอินของ AI agent ที่มีทั้ง Web UI และ REST API ให้นักพัฒนาเผยแพร่เครื่องมือและเวอร์ชัน ให้ผู้ดูแลอนุมัติก่อนเผยแพร่ตามวงจร DRAFT ⟶ PENDING ⟶ PUBLISHED ⟶ DEPRECATED และให้ผู้ใช้ค้นหา กรอง จัดเรียง ให้คะแนน และรีวิว ระบบแบ่งงานตาม Controller, Service, Repository และ Domain/Entity ใช้ DTO/Mapper แยก API contract จาก persistence และใช้ PostgreSQL 8 ตารางที่มีความสัมพันธ์ one-to-one, one-to-many และ many-to-many

ในส่วนการออกแบบ ใช้ Strategy สำหรับการเรียงผลค้นหา State สำหรับวงจรการเผยแพร่ และ Observer สำหรับ audit หลัง commit ส่วน SOLID มีการประยุกต์หลายส่วน แต่ยังไม่ควรสรุปว่าผ่านครบ โดยเฉพาะ OCP, SRP และ DIP รวมถึงการพิสูจน์ LSP/ISP ด้วย contract tests [2]

4.2 ข้อจำกัด

(1)ไม่มี OAuth การยืนยันอีเมล หรือการกู้คืนรหัสผ่าน และมีบทบาทผู้ดูแลเพียงระดับเดียว

(2)ระบบเก็บเฉพาะข้อมูลอธิบายและลิงก์ของเครื่องมือ ไม่ได้รันหรือตรวจความปลอดภัยของโค้ด การอนุมัติเป็นการตรวจโดยผู้ดูแลด้วยตนเอง

(3)ผลทดสอบ 736 รายการเป็นผลที่บันทึกไว้ที่ SHA 2af3802 ไม่ใช่ผลยืนยันของ commit ล่าสุด

(4)Data Dictionary ใน repository ยังสรุปคอลัมน์ไม่ครบทุกฟิลด์ (ภาคผนวก ก ของรายงานนี้สรุปจาก schema.sql)

(5)ใช้บริการระดับฟรีของผู้ให้บริการคลาวด์ จึงมีข้อจำกัดด้านทรัพยากรและจำนวนผู้ใช้พร้อมกัน

4.3 ข้อเสนอแนะ

1) แยกนโยบายการเรียงตามคะแนน (rating) และความเกี่ยวข้อง (relevance) เป็น strategy พร้อมรักษา tie-break และ pagination semantics
2) แยก version normalization และ response mapping ออกจาก ToolVersionServiceImpl โดยคง business invariants และ transaction
3) เพิ่ม contract tests สำหรับ ToolSortStrategy และ PublishingState รวมทั้ง invalid transition และสิทธิ์ owner/admin

4) เติม data dictionary ใน repository ให้ครบทุกฟิลด์ พร้อมเหตุผลของ cascade/fetch และดัชนี

5) ต่อยอดด้วย OAuth การตรวจความปลอดภัยของเครื่องมืออัตโนมัติ และการเชื่อมต่อกับมาตรฐาน MCP โดยตรง

4.4 บทสรุป

PrimeSkill ตอบโจทย์การรวมศูนย์ค้นหา เผยแพร่ และรีวิวเครื่องมือของ AI agent พร้อมกระบวนการอนุมัติเพื่อเพิ่มความน่าเชื่อถือ และเป็นตัวอย่างการประยุกต์สถาปัตยกรรมแบบแบ่งชั้น SOLID และ GoF Behavioral Patterns ในระบบที่ติดตั้งใช้งานได้จริง งานต่อไปควรเน้นลดข้อจำกัดด้าน SOLID ที่ทราบ และขยายความสามารถตามข้อเสนอแนะข้างต้น

เอกสารอ้างอิง

[1] คณะผู้สอนรายวิชา CP353002. (ม.ป.ป.). ใบงานโปรเจค CP353002 Principles of Software Design and Development (Spring Boot). เอกสารประกอบรายวิชา.

[2] aarktik. (2569). PrimeSkill: Source code and project documentation [GitHub repository]. GitHub. https://github.com/aarktik/PrimeSkill (สืบค้น 11 ตุลาคม 2569; branch main, commit 5b1267b).

[3] Anthropic. (2024). Model Context Protocol. https://modelcontextprotocol.io

[4] Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

[5] Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation]. University of California, Irvine.

[6] OpenAPI Initiative. (n.d.). OpenAPI Specification. https://spec.openapis.org/oas/latest.html

[7] Spring. (2569). Spring Boot Reference Documentation, version 4.1.1. https://docs.spring.io/spring-boot/reference/ (สืบค้น 11 ตุลาคม 2569).

[8] Spring. (2569). Spring Data JPA Reference Documentation, version 4.1.1. https://docs.spring.io/spring-data/jpa/reference/ (สืบค้น 11 ตุลาคม 2569).

[9] OWASP Foundation. (n.d.). OWASP Top Ten. https://owasp.org/www-project-top-ten/

[10] PostgreSQL Global Development Group. (2569). PostgreSQL 18 Documentation: Constraints. https://www.postgresql.org/docs/current/ddl-constraints.html (สืบค้น 11 ตุลาคม 2569).

[11] Martin, R. C. (2003). Agile software development: Principles, patterns, and practices. Prentice Hall.

[12] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

[13] Docker Inc. (n.d.). Docker documentation. https://docs.docker.com

ภาคผนวก ก พจนานุกรมข้อมูล

สรุปชนิดข้อมูลและข้อกำหนดจาก code/src/main/resources/schema.sql โดยแสดง 8 ตารางธุรกิจ ไม่รวมตาราง infrastructure ของ Spring Session

users

Column	Type	Key / rule
id	BIGINT	PK; identity
email	VARCHAR(255)	NOT NULL; UNIQUE
password_hash	VARCHAR(255)	NOT NULL
role	VARCHAR(50)	NOT NULL; default USER
enabled	BOOLEAN	NOT NULL; default TRUE
created_at	TIMESTAMPTZ	NOT NULL; default current timestamp
updated_at	TIMESTAMPTZ	NOT NULL; default current timestamp

user_profiles

Column	Type	Key / rule
id	BIGINT	PK; identity
user_id	BIGINT	FK users.id; UNIQUE; NOT NULL; CASCADE
display_name	VARCHAR(100)	NOT NULL
bio	TEXT	nullable
avatar_url	VARCHAR(2048)	nullable
created_at	TIMESTAMPTZ	NOT NULL; default current timestamp
updated_at	TIMESTAMPTZ	NOT NULL; default current timestamp

categories

Column	Type	Key / rule
id	BIGINT	PK; identity
name	VARCHAR(100)	NOT NULL; UNIQUE
slug	VARCHAR(120)	NOT NULL; UNIQUE
description	VARCHAR(500)	nullable
created_at	TIMESTAMPTZ	NOT NULL; default current timestamp
updated_at	TIMESTAMPTZ	NOT NULL; default current timestamp

tools

Column	Type	Key / rule
id	BIGINT	PK; identity
name	VARCHAR(150)	NOT NULL
slug	VARCHAR(170)	NOT NULL; UNIQUE
short_description	VARCHAR(300)	NOT NULL
description	TEXT	NOT NULL
repository_url	VARCHAR(500)	nullable
category_id	BIGINT	FK categories.id; NOT NULL; RESTRICT; index
owner_id	BIGINT	FK users.id; NOT NULL; index
status	VARCHAR(30)	NOT NULL; default DRAFT; index
review_revision	BIGINT	NOT NULL; default 0; CHECK >= 0
view_count	BIGINT	NOT NULL; default 0
created_at	TIMESTAMPTZ	NOT NULL; default current timestamp
updated_at	TIMESTAMPTZ	NOT NULL; default current timestamp

tags

Column	Type	Key / rule
id	BIGINT	PK; identity
name	VARCHAR(100)	NOT NULL; UNIQUE
slug	VARCHAR(120)	NOT NULL; UNIQUE
created_at	TIMESTAMPTZ	NOT NULL; default current timestamp
updated_at	TIMESTAMPTZ	NOT NULL; default current timestamp

tool_tags

Column	Type	Key / rule
tool_id	BIGINT	Composite PK; FK tools.id; CASCADE; index
tag_id	BIGINT	Composite PK; FK tags.id; CASCADE; index

reviews

Column	Type	Key / rule
id	BIGINT	PK; identity
user_id	BIGINT	FK users.id; NOT NULL; CASCADE
tool_id	BIGINT	FK tools.id; NOT NULL; CASCADE; index
rating	SMALLINT	NOT NULL; CHECK 1–5
comment	TEXT	nullable; max 2,000 chars
created_at	TIMESTAMPTZ	NOT NULL; default current timestamp
updated_at	TIMESTAMPTZ	NOT NULL; default current timestamp
(user_id, tool_id)	UNIQUE	หนึ่งรีวิวต่อผู้ใช้ต่อเครื่องมือ

tool_versions

Column	Type	Key / rule
id	BIGINT	PK; identity
tool_id	BIGINT	FK tools.id; NOT NULL; CASCADE
version	VARCHAR(100)	NOT NULL
release_notes	TEXT	nullable
released_at	TIMESTAMPTZ	nullable
created_at	TIMESTAMPTZ	NOT NULL; default current timestamp
updated_at	TIMESTAMPTZ	NOT NULL; default current timestamp
(tool_id, version)	UNIQUE	ห้าม version ซ้ำใน tool เดียวกัน